UNITÀ 4 · APPLICAZIONI PER SISTEMI MOBILI · LEZIONE 6

L'interattività: Button e Toast

QuizTPSI impara ad ascoltare: pulsanti Vero e Falso, un messaggio che compare e sparisce, i punti che crescono. E un bug che tutti incontrano almeno una volta: la rotazione.

5G TPSI Eventi, Button, setOnClickListener, Toast

PER COMINCIARE

Dopo un tocco, quale conferma preferisci?

SONDAGGIO

- **A** Una breve vibrazione
- **B** Un messaggio che compare e sparisce
- **C** Il pulsante che cambia colore
- **D** Un suono
- **E** Nessuna: mi basta che funzioni

PROGRAMMAZIONE A EVENTI

L'app non fa niente: aspetta

Evento

Il dito tocca lo schermo, arriva una notifica, scade un timer.

Main thread

Un unico thread prende gli eventi da una coda, uno alla volta.

Listener

Il tuo codice, registrato prima, viene chiamato.

Ridisegno

Lo stesso thread aggiorna l'interfaccia.

> Se un listener lavora troppo a lungo il thread è bloccato: dopo circa **5 secondi** senza risposta a un input, Android mostra la finestra **ANR**, "L'app non risponde".

IL PRIMO PULSANTE

Un Button nel layout, un listener nel codice

```xml
<Button
    android:id="@+id/btnVero"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/vero" />
```

```kotlin
val btnVero =
    findViewById<Button>(R.id.btnVero)

btnVero.setOnClickListener {
    Toast.makeText(this, "Hai detto Vero",
        Toast.LENGTH_SHORT).show()
}
```

In Java si scriveva `btnVero.setOnClickListener(v -> { … });` o una classe anonima che implementa `View.OnClickListener`.

In Kotlin la lambda finale va fuori dalle parentesi; `this` è l'Activity, che fa da **Context** per il Toast.

QUIZ A TRABOCCHETTO

Nel listener c'è solo questa riga. Tocchi il pulsante: che cosa succede?

```
Toast.makeText(this, "Giusto!", Toast.LENGTH_SHORT)
```

QUIZ

A Compare il messaggio "Giusto!"

B Il progetto non compila

C Non compare niente, e non c'è nessun errore

D L'app si chiude con un'eccezione

AL COMPUTER · 20 minuti

I pulsanti Vero e Falso

1. In strings.xml aggiungi `vero`, `falso`, `giusto` e `sbagliato`.
2. Sotto tvDomanda aggiungi un LinearLayout **orizzontale** con **btnVero** e **btnFalso**.
3. Dai a entrambi `layout_width="0dp"` e `layout_weight="1"`.
4. In onCreate registra un listener su ciascuno che mostra un Toast.

```
btnFalso.setOnClickListener {
    Toast.makeText(this, R.string.sbagliato,
        Toast.LENGTH_SHORT).show()
}
```

Controllo: i due pulsanti dividono a metà la larghezza dello schermo. Toccando Vero compare **Giusto!**, toccando Falso compare **Sbagliato** (la domanda 1 è vera).

IL TOAST

Un messaggio breve, che non si tocca

- `LENGTH_SHORT` dura circa 2 secondi, `LENGTH_LONG` circa 3,5.
- Non ha pulsanti e non si può toccare: solo informazione.
- Da Android 12 un Toast di testo mostra al massimo due righe, con l'icona dell'app.
- Per un'azione da annullare si usa una **Snackbar**; per una scelta obbligata un **dialog**.

Toast o Logcat?

Il Toast lo vede l'utente. Per capire che cosa fa il codice usa `Log.d`: un Toast messo "per debug" e dimenticato finisce in produzione.

PUZZLE

Dal dito al Toast: metti in ordine

1. Sul main thread il Button riconosce un clic completo
2. Il sistema crea un evento di tocco e lo consegna alla finestra dell'app
3. Il Button chiama la lambda registrata con setOnClickListener
4. Il codice crea il Toast con makeText
5. show() chiede al sistema di mostrarlo sopra l'app
6. Il dito tocca lo schermo sopra btnVero

LA LOGICA DEL QUIZ

Dati in una data class, stato in due variabili

```
data class Domanda(val testo: Int, val immagine: Int, val risposta: Boolean)

private val domande = listOf(
    Domanda(R.string.domanda_1, R.drawable.img_domanda1, true),
    Domanda(R.string.domanda_2, R.drawable.img_domanda2, false),
    Domanda(R.string.domanda_3, R.drawable.img_domanda3, false))
private var indice = 0
private var punti = 0

private fun controlla(rispostaUtente: Boolean) {
    val giusta = rispostaUtente == domande[indice].risposta
    if (giusta) punti++
    Toast.makeText(this, if (giusta) R.string.giusto else R.string.sbagliato,
        Toast.LENGTH_SHORT).show()
}
```

AL COMPUTER · 25 minuti

Il quiz funziona davvero

1. Aggiungi `domanda_2`, `domanda_3` e le immagini `img_domanda2`, `img_domanda3`.
2. Copia la data class e la funzione `controlla`; i listener chiamano `controlla(true)` e `controlla(false)`.
3. Scrivi `mostraDomanda()`: aggiorna testo, immagine e contatore da `domande[indice]`.
4. Aggiungi **btnAvanti**: se ci sono altre domande, `indice++` e `mostraDomanda()`; altrimenti un Toast con i punti.

Controllo: rispondendo Vero, Falso, Falso, all'ultimo Avanti compare **Quiz finito: 3 su 3**. Il contatore passa da Domanda 1 di 3 a Domanda 3 di 3.

METTITI ALLA PROVA

Hai risposto a due domande, punti = 2. Ruoti il telefono: che cosa vedi?

QUIZ

A Domanda 3 di 3, con 2 punti

B Domanda 2 di 3, con 2 punti

C L'app si chiude: errore di indice

D Domanda 1 di 3, con 0 punti

LA CORREZIONE

Salva lo stato prima che l'Activity sparisca

```
override fun onSaveInstanceState(outState: Bundle) {
    super.onSaveInstanceState(outState)
    outState.putInt("indice", indice)
    outState.putInt("punti", punti)
}

// in onCreate, prima di mostraDomanda()
indice = savedInstanceState?.getInt("indice") ?: 0
punti = savedInstanceState?.getInt("punti") ?: 0
```

- Il **Bundle** è una mappa chiave-valore che il sistema conserva.
- Al primo avvio `savedInstanceState` è null: ecco perché `?.` e `?:`.
- Per stati più grandi si usa un **ViewModel**, che sopravvive alla rotazione.

AL COMPUTER · 15 minuti

Niente più punti persi, niente più punti regalati

1. Aggiungi `onSaveInstanceState` e il ripristino in onCreate, come nella slide.
2. In `controlla`, dopo la risposta, disattiva i pulsanti con `isEnabled = false`.
3. In `mostraDomanda` riattivali.
4. Salva anche se la domanda corrente ha già una risposta, e ripristinalo.

Controllo: rispondi a due domande, ruota: resti su **Domanda 2 di 3** con i punti giusti. Toccare Vero tre volte dà al massimo un punto.

MENTI CHE CI HANNO PRECEDUTO

Gli utenti toccheranno tutto

Apollo 8, dicembre 1968

Margaret Hamilton, a capo del software di bordo al MIT, aveva visto la figlia avviare per gioco il programma P01 nel simulatore, cancellando i dati di navigazione. Le risposero che gli astronauti, addestrati, non l'avrebbero mai fatto. In volo Jim Lovell lo fece. Da allora il software di Apollo ebbe più controlli sugli errori dell'utente.

> **Il collaudo di un programma può mostrare la presenza di bug, mai la loro assenza.**
>
> **Edsger W. Dijkstra**, informatico olandese, Notes on Structured Programming, 1970

PUZZLE

Abbina ogni termine al suo significato

setOnClickListener	---	L'app non risponde a un input da circa 5 secondi
Toast.LENGTH_LONG	---	Messaggio visibile per circa 3,5 secondi
isEnabled = false	---	Riceve gli eventi e disegna l'interfaccia
onSaveInstanceState	---	Il pulsante non risponde più ai tocchi
Main thread	---	Salva lo stato prima che l'Activity venga distrutta
ANR	---	Registra il codice da eseguire al clic

I PULSANTI OGGI

Un pulsante può chiedere aiuto all'AI, o essere una trappola





BIGLIETTO D'USCITA

Completa prima di uscire

Il codice da eseguire al clic si registra con _____.

Senza la chiamata a _____ il Toast non compare.

Gli eventi e il disegno dell'interfaccia passano dal _____.

Lo stato da conservare alla rotazione si salva in onSaveInstanceState dentro un _____.

Un pulsante si disattiva con `isEnabled =` _____.

⏱ 3:00

RIEPILOGO

Cosa portiamo a casa

- L'app reagisce a eventi, sul main thread
- setOnClickListener { … } con una lambda
- Toast.makeText(…).show(): show non si dimentica
- Dati in data class, stato in variabili
- La rotazione cancella lo stato: onSaveInstanceState
- Prevenire gli errori dell'utente: isEnabled

Hai una domanda su una slide? Usa il pulsante **? Domande** in basso: la vedrà il docente e tutta la classe.